WINPMEM FORENSICS LAB

Acquisition Preparation & SHA-256 Integrity Verification

Sergio Alejandro Perez | ITDF-2420-MY1 | October 24, 2026

Project overview

This educational lab documents preparation of the WinPmem utility in a Windows environment and verification of a WinPmem executable's SHA-256 hash using two built-in Windows interfaces: Command Prompt (CertUtil) and PowerShell (Get-FileHash). The work demonstrates directory navigation, file handling, command-line execution, and comparison of hash outputs.

Tool / platform	WinPmem files, Windows Command Prompt, Windows PowerShell
Evidence shown	Instructor-provided ZIP, extracted files, commands, matching SHA-256 output
Scope	Preparation and executable hash comparison; RAM capture not documented

01 Prepare the WinPmem files

I downloaded the WinPmem ZIP supplied for the course, extracted its contents, and organized the archive and extracted folder in a working directory. This kept the executable accessible for command-line operations.

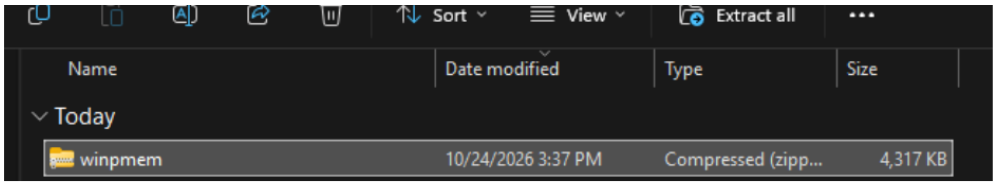


Figure 1. Downloaded WinPmem archive.



Figure 2. Archive and extracted directory.

02 Navigate with an elevated Command Prompt

I opened Command Prompt as administrator and navigated to the directory containing the WinPmem files. The original screenshot shows the working path **C:\Evidence\winpmem**.

```
cd C:\Evidence\winpmem
```

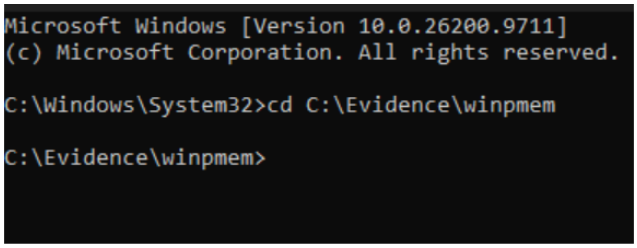


Figure 3. Elevated Command Prompt navigation.

03 Select the executable

I located the extracted WinPmem executable and used Windows File Explorer's **Copy as path** option to capture its full path for hashing. The source screenshot shows the WinPmem mini executables.

one of them as a path which will look something like this:

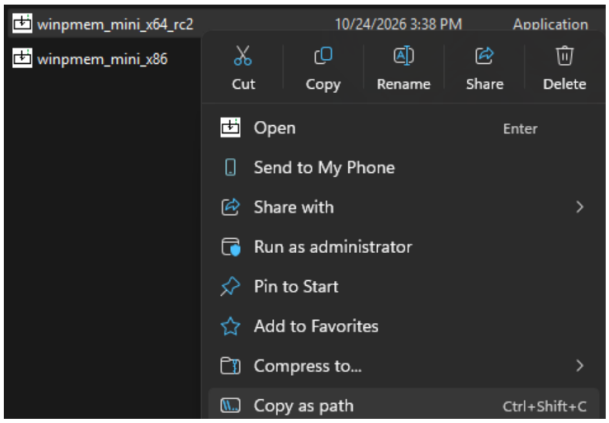


Figure 4. WinPmem executable and Copy as path.

04 Calculate and compare SHA-256 hashes

I generated a SHA-256 hash for the selected executable using **CertUtil** in Command Prompt and **Get-FileHash** in PowerShell. The screenshot shows the same SHA-256 output for the selected file in both tools.

```
CertUtil -hashfile "C:\Evidence\winpmem\winpmem_mini_x64_rc2.exe" SHA256
```

```
Get-FileHash "C:\Evidence\winpmem\winpmem_mini_x64_rc2.exe" -Algorithm SHA256
```

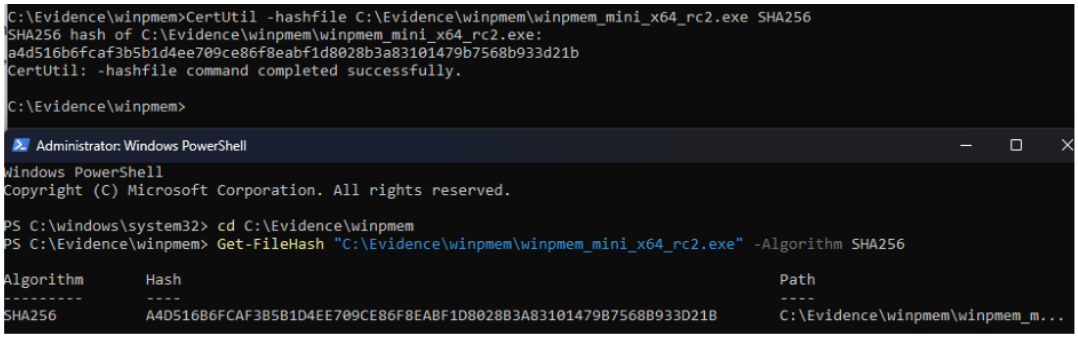


Figure 5. SHA-256 outputs from CertUtil and PowerShell.

Interpretation: Matching values demonstrate that the two hashing methods produced the same digest for the file. This is a useful consistency check, but it does **not** independently prove that the executable is authentic or uncompromised without comparison to a trusted reference hash.

Skills demonstrated

• Prepared and organized forensic utility files in Windows.

• Used elevated Command Prompt and navigated directories with cd.

• Identified a file path and ran hashing commands.

• Compared SHA-256 outputs between CertUtil and PowerShell.

• Documented a reproducible introductory digital forensics workflow.

Reflection and next steps

The lab reinforced the value of verifying files with cryptographic hashes and showed how equivalent results can be obtained through different Windows command-line tools. A logical next exercise would be a controlled RAM acquisition using WinPmem on an authorized test machine, followed by documentation of the capture command, output size, acquisition time, hash of the resulting memory image, and analysis in an appropriate forensic tool.

Project limitations

Although the original coursework is titled “WinPmem - Memory Capture,” the submitted pages do not show execution of a RAM acquisition command, a generated memory image, or analysis of volatile memory. This portfolio therefore describes the verified work as **memory-acquisition preparation and file hashing**, not a completed memory capture. Original coursework screenshots have been retained.